

GCP Full Lab Manual — Networking 1.1–1.3 + Terraform IAM/GCE/SQL + Ansible PostgreSQL

This book starts at networking (including Private Service Access). Then Terraform modules for IAM, Compute Engine, Cloud SQL. Then Ansible installs PostgreSQL on the VM.

If you already ran gcloud 1.1–1.3, do not create a second VPC or second PSA range. Use Lab 1.4 import / skip. Never open 5432 to 0.0.0.0/0.

0. Project and APIs

```
export PROJECT_ID="$(gcloud config get-value project)"
export REGION=asia-south1
export ZONE=asia-south1-a
echo "$PROJECT_ID"
gcloud services enable compute.googleapis.com sqladmin.googleapis.com iam.googleapis.com \
  servicenetworking.googleapis.com cloudresourcemanager.googleapis.com
mkdir -p ~/tf-ansible-pg/{modules/{network,iam,compute,sql},ansible/{playbooks,inventories},env}
cd ~/tf-ansible-pg
```

1.1 Create VPC and subnet

gcloud (do this if the VPC does not exist yet):

```
gcloud compute networks create pg-lab-vpc --subnet-mode=custom --bgp-routing-mode=regional
gcloud compute networks subnets create pg-lab-subnet \
  --network=pg-lab-vpc --region="$REGION" --range=10.20.0.0/24 --enable-private-ip-google-access
gcloud compute networks describe pg-lab-vpc --format='get(selfLink)'
gcloud compute networks subnets describe pg-lab-subnet --region="$REGION" --format='get(ipCidrRange)'
```

Expected: network exists; subnet 10.20.0.0/24; privateIpGoogleAccess true.

1.2 Firewall — IAP SSH and Postgres from subnet only

```
gcloud compute firewall-rules create pg-lab-allow-iap-ssh \
  --network=pg-lab-vpc --allow-tcp:22 --source-ranges=35.235.240.0/20 --target-tags=pg-lab

gcloud compute firewall-rules create pg-lab-allow-5432-internal \
  --network=pg-lab-vpc --allow-tcp:5432 --source-ranges=10.20.0.0/24 --target-tags=pg-lab

gcloud compute firewall-rules list --filter="network=pg-lab-vpc"
```

35.235.240.0/20 is IAP. Do not use 0.0.0.0/0 on 22 or 5432.

1.3 Private Service Access for Cloud SQL (required)

Cloud SQL private IP needs a reserved range and VPC peering to servicenetworking.googleapis.com. Skip the create commands if gcloud already printed that peering exists.

```
# reserved range for Google-managed services (Cloud SQL)
gcloud compute addresses create pg-lab-psa --global --purpose=VPC_PEERING \
  --prefix-length=16 --network=pg-lab-vpc

gcloud compute addresses describe pg-lab-psa --global

gcloud services vpc-peerings connect --service=servicenetworking.googleapis.com \
  --ranges=pg-lab-psa --network=pg-lab-vpc

gcloud services vpc-peerings list --network=pg-lab-vpc
```

If connect says already exists, continue. Do not allocate a second /16 on the same VPC.

1.3 check-off: address pg-lab-psa exists; peering to servicenetworking.googleapis.com is CONNECTED.

1.4 Same networking in Terraform (module + import)

```
cat > env/terraform.tfvars <<EOF
project_id    = "${PROJECT_ID}"
region        = "asia-south1"
zone          = "asia-south1-a"
network_name  = "pg-lab-vpc"
subnet_name   = "pg-lab-subnet"
subnet_cidr   = "10.20.0.0/24"
vm_name       = "pg-ansible-vm"
sql_name      = "pg-lab-sql"
sql_password  = "ChangeMe-Lab-Only"
create_network = false
EOF
```

`create_network=false` because 1.1–1.3 already created the VPC. Set true only on a greenfield project (then still run 1.3 PSA — TF snippet below).

```
cat > modules/network/main.tf <<'EOF'
variable "create_network" { type = bool }
variable "network_name" { type = string }
variable "subnet_name" { type = string }
variable "subnet_cidr" { type = string }
variable "region" { type = string }

resource "google_compute_network" "this" {
  count          = var.create_network ? 1 : 0
  name           = var.network_name
  auto_create_subnetworks = false
}
data "google_compute_network" "this" {
  name = var.network_name
}
data "google_compute_subnetwork" "this" {
  name     = var.subnet_name
  region   = var.region
}

resource "google_compute_global_address" "psa" {
  name          = "pg-lab-psa"
  purpose       = "VPC_PEERING"
  address_type  = "INTERNAL"
  prefix_length = 16
  network       = data.google_compute_network.this.id
}
resource "google_service_networking_connection" "psa" {
  network                 = data.google_compute_network.this.id
  service                 = "servicenetworking.googleapis.com"
  reserved_peering_ranges = [google_compute_global_address.psa.name]
}

resource "google_compute_firewall" "iap_ssh" {
  name     = "pg-lab-allow-iap-ssh"
  network  = data.google_compute_network.this.name
  allow { protocol = "tcp"; ports = ["22"] }
  source_ranges = ["35.235.240.0/20"]
  target_tags  = ["pg-lab"]
}
resource "google_compute_firewall" "pg" {
  name     = "pg-lab-allow-5432-internal"
  network  = data.google_compute_network.this.name
  allow { protocol = "tcp"; ports = ["5432"] }
  source_ranges = [var.subnet_cidr]
  target_tags  = ["pg-lab"]
}

output "network_id" { value = data.google_compute_network.this.id }
output "subnet_self_link" { value = data.google_compute_subnetwork.this.self_link }
EOF
```

Import objects that 1.1–1.3 already created so apply does not fight gcloud:

```
# after terraform init (Lab 6)
terraform import -var-file=env/terraform.tfvars module.network.google_compute_global_address.psa pg-lab-psa
terraform import -var-file=env/terraform.tfvars \
  'module.network.google_service_networking_connection.psa' \
  'projects/YOUR_PROJECT/global/networks/pg-lab-vpc:servicenetworking.googleapis.com'
terraform import -var-file=env/terraform.tfvars module.network.google_compute_firewall.iap_ssh pg-lab-allow-iap-ssh
terraform import -var-file=env/terraform.tfvars module.network.google_compute_firewall.pg pg-lab-allow-5432-internal
```

2. Terraform skeleton (versions, variables, root)

```
cat > versions.tf <<'EOF'
terraform {
  required_version = ">= 1.6.0"
  required_providers {
    google = { source = "hashicorp/google", version = "~> 5.0" }
  }
}
provider "google" { project = var.project_id; region = var.region }
EOF
cat > variables.tf <<'EOF'
variable "project_id" { type = string }
variable "region" { type = string }
variable "zone" { type = string }
variable "network_name" { type = string }
variable "subnet_name" { type = string }
variable "subnet_cidr" { type = string }
variable "vm_name" { type = string }
variable "sql_name" { type = string }
```

```
variable "sql_password" { type = string; sensitive = true }
variable "create_network" { type = bool; default = false }
EOF
```

3. IAM module

```
cat > modules/iam/main.tf <<'EOF'
variable "project_id" { type = string }
resource "google_service_account" "vm" { account_id = "pg-vm-sa"; display_name = "PostgreSQL VM" }
resource "google_service_account" "sql" { account_id = "pg-sql-client-sa"; display_name = "Cloud SQL client" }
resource "google_project_iam_member" "vm_log" {
  project = var.project_id; role = "roles/logging.logWriter"
  member  = "serviceAccount:${google_service_account.vm.email}"
}
resource "google_project_iam_member" "vm_mon" {
  project = var.project_id; role = "roles/monitoring.metricWriter"
  member  = "serviceAccount:${google_service_account.vm.email}"
}
resource "google_project_iam_member" "sql_client" {
  project = var.project_id; role = "roles/cloudsql.client"
  member  = "serviceAccount:${google_service_account.sql.email}"
}
output "vm_email" { value = google_service_account.vm.email }
output "sql_email" { value = google_service_account.sql.email }
EOF
```

4. Compute Engine module

```
cat > modules/compute/main.tf <<'EOF'
variable "name" { type = string }
variable "zone" { type = string }
variable "subnet" { type = string }
variable "sa_email" { type = string }
data "google_compute_image" "ubuntu" {
  family  = "ubuntu-2404-lts-amd64"
  project = "ubuntu-os-cloud"
}
resource "google_compute_instance" "vm" {
  name          = var.name
  machine_type  = "e2-medium"
  zone          = var.zone
  tags          = ["pg-lab"]
  boot_disk { initialize_params { image = data.google_compute_image.ubuntu.self_link; size = 20; type = "pd-balanced" } }
  network_interface { subnetwork = var.subnet }
  service_account { email = var.sa_email; scopes = ["cloud-platform"] }
  metadata = { enable-oslogin = "TRUE" }
}
output "internal_ip" { value = google_compute_instance.vm.network_interface[0].network_ip }
EOF
```

5. Cloud SQL module (private IP — needs 1.3)

```
cat > modules/sql/main.tf <<'EOF'
variable "name" { type = string }
variable "region" { type = string }
variable "network_id" { type = string }
variable "password" { type = string; sensitive = true }
resource "google_sql_database_instance" "this" {
  name             = var.name
  database_version = "POSTGRES_16"
  region           = var.region
  settings {
    tier = "db-custom-1-3840"
    ip_configuration { ipv4_enabled = false; private_network = var.network_id }
    availability_type = "ZONAL"
    disk_size = 20
  }
  root_password = var.password
}
resource "google_sql_database" "appdb" { name = "appdb"; instance = google_sql_database_instance.this.name }
resource "google_sql_user" "appuser" { name = "appuser"; instance = google_sql_database_instance.this.name; password = var.password }
output "connection_name" { value = google_sql_database_instance.this.connection_name }
output "private_ip" { value = google_sql_database_instance.this.private_ip_address }
EOF
```

6. Root module, plan, apply

```
cat > main.tf <<'EOF'
module "network" {
  source = "../modules/network"
  create_network = var.create_network
  network_name = var.network_name
}
```

```

    subnet_name = var.subnet_name
    subnet_cidr = var.subnet_cidr
    region      = var.region
  }
  module "iam" { source = "../modules/iam"; project_id = var.project_id }
  module "compute" {
    source = "../modules/compute"
    name = var.vm_name; zone = var.zone
    subnet = module.network.subnet_self_link
    sa_email = module.iam.vm_email
  }
  module "sql" {
    source = "../modules/sql"
    name = var.sql_name; region = var.region
    network_id = module.network.network_id
    password = var.sql_password
    depends_on = [module.network]
  }
  output "vm_ip" { value = module.compute.internal_ip }
  output "sql_connection" { value = module.sql.connection_name }
  output "sql_private_ip" { value = module.sql.private_ip }
EOF
terraform init
terraform import ... # Lab 1.4 if resources exist
terraform plan -var-file=env/terraform.tfvars -out=tfplan
terraform apply tfplan
terraform output

```

7. Ansible — PostgreSQL on the VM

```

VM_IP=$(terraform output -raw vm_ip)
cat > ansible/inventories/lab.ini <<EOF
[postgres]
pg-ansible-vm ansible_host=${VM_IP} ansible_user=$(whoami) ansible_python_interpreter=/usr/bin/python3
EOF
gcloud compute ssh pg-ansible-vm --zone="$ZONE" --tunnel-through-iap --command='echo ok'
cat > ansible/playbooks/postgres.yml <<'EOF'
---
- hosts: postgres
  become: true
  vars: { pg_version: 16 }
  tasks:
    - apt: { name: ["postgresql-{{ pg_version }}", "postgresql-contrib-{{ pg_version }}", "python3-psycopg2"], update_cache: true }
    - lineinfile: { path: "/etc/postgresql/{{ pg_version }}/main/postgresql.conf", regexp: "^#?listen_addresses", line: "listen_addresses = "
      notify: reload postgres
    - lineinfile: { path: "/etc/postgresql/{{ pg_version }}/main/pg_hba.conf", line: "host all all 10.20.0.0/24 scram-sha-256" }
      notify: reload postgres
    - file: { path: /var/backups/postgresql, state: directory, owner: postgres, mode: "0750" }
    - become_user: postgres
      community.postgresql.postgresql_user: { name: appuser, password: "ChangeMe-Lab-Only", role_attr_flags: LOGIN }
    - become_user: postgres
      community.postgresql.postgresql_db: { name: appdb, owner: appuser }
  handlers:
    - name: reload postgres
      service: { name: postgresql, state: reloaded }
EOF
ansible-galaxy collection install community.postgresql
# local on VM if IAP-from-Ansible is awkward:
# ansible-playbook -i 'localhost,' -c local ansible/playbooks/postgres.yml
ansible-playbook -i ansible/inventories/lab.ini ansible/playbooks/postgres.yml

```

8. Verify

```

gcloud compute ssh pg-ansible-vm --zone="$ZONE" --tunnel-through-iap --command='
  sudo -u postgres psql -c "SELECT version();"
  sudo -u postgres pg_dump -Fc appdb -f /var/backups/postgresql/appdb.dump
  ls -l /var/backups/postgresql/
'
gcloud sql instances describe pg-lab-sql --format='get(ipAddresses)'
terraform output

```

9. Cleanup

```

terraform destroy -var-file=env/terraform.tfvars
# data-source VPC is not destroyed. To remove 1.1-1.3 leftovers:
# gcloud sql instances delete pg-lab-sql --quiet
# gcloud compute instances delete pg-ansible-vm --zone=$ZONE --quiet
# gcloud services vpc-peerings delete --service=servicenetworking.googleapis.com --network=pg-lab-vpc --quiet
# gcloud compute addresses delete pg-lab-psa --global --quiet
# gcloud compute firewall-rules delete pg-lab-allow-iap-ssh pg-lab-allow-5432-internal --quiet
# gcloud compute networks subnets delete pg-lab-subnet --region=$REGION --quiet
# gcloud compute networks delete pg-lab-vpc --quiet

```

Step	Covered
1.1 VPC + subnet	gcloud + TF data/resource
1.2 Firewalls	gcloud + TF + import
1.3 PSA range + peering	gcloud + TF google_compute_global_address + google_service_networking_connection + import
IAM	TF module iam
Compute Engine	TF module compute, no public IP
Cloud SQL	TF module sql, private IP only
Ansible PostgreSQL	playbook install/config/user/dump

Pass: 1.3 peering CONNECTED; terraform apply (after import) adds VM+SQL+SAs; Ansible leaves pg_dump on the VM; no public IPs.